E-commerce Non-buyer Detection

Technical Data Analyst Report

Prepared By: Rihem Abdelmoumen

Major:Business Analytics | Minor: Information Technology

October 2026

Report Navigation

1. Executive Summary……..…....3

2. Business Context, Scope and Research Questions 4

2.1 Illustrative Organizational Context…..... .4
2.2 Business Question and Decision Boundaries..... ..4
2.3 Objectives……..……..…….……..... .5
2.4 Research Questions…….…….....5

3. Dataset and Exploratory Assessment……..…….…..…..…...5

3.1 Source, Target and Unit of Analysis……..……..... .5
3.2 Data Quality and Cleaning Decisions..... ..7
3.3 Extreme Imbalance and Descriptive Profiles……..... .8

4. Methodology……..…….…..11

4.1 Workflow Overview..... .9
4.2 Leakage Prevention……..... ..9
4.3 Tools and Configuration……..... .11
4.4 Data Preparation and EDA……..……....12
4.5 Preprocessing and Feature Schema…….……..... 14
4.6 Model Configurations and Validation Design……..……..... ...16
4.7 Evaluation Metrics and Convention…….....20
4.8 Out-of Fold and Threshold Analysis ……..….. ...22
4.9 Sensitivity Analysis…….... 26
4.10 Model Interpretation…….…….....27
4.11 Final Model……..…….…….…… 28
4.12 Exploratory Principal Component Analysis……..…….. 30
4.13 Output Management and Reporting…….……...…….32

5. Results and Interpretations…….…..…..…….33

5.1 Cross-validated Model Comparaison……..... ..33
5.2 Threshold and Operating point Analysis……... …36
5.3 Sensitivity Analysis……..... .38
5.4 Logistic Regression Coefficients.....40
5.5 Tree-based Interpretation ……..……..... 42
5.6 PCA Results……..……..... ...44

6. Key Findings……..... ..47

7. Recommendations 48

8. Limitations……..... 50

9. Conclusion 51

Dataset	Kaggle, `ecommerce_user_behavior_8000.csv` ; simulated browsing sessions
Evaluation	Repeated stratified 5-fold cross-validation, 20 repeats = 100 training/validation splits
Models	Logistic Regression, Decision Tree, Random Forest
Software	Python 3, pandas 3.0.2, NumPy 2.4.4, scikit-learn 1.8, imbalanced-learn 0.14.2

1. Executive Summary

E-commerce platforms record session-level signals such as time on site, pages viewed, cart activity, previous purchases, device type, and promotional exposure. The project evaluates whether session-level behaviour can distinguish purchasing from non-purchasing sessions in a simulated e-commerce dataset.

The modelling population contains 7,840 labelled records: 7,827 buyers and only 13 non-buyers (0.166%, approximately 0.17%). A constant buyer prediction would achieve 99.83% accuracy but detect no non-buyers. Rare-class ranking and detection, rather than accuracy, therefore define success.

Logistic Regression, Decision Tree and Random Forest were compared using stratified five-fold cross-validation repeated 20 times. Imputation, scaling, encoding and potential resampling were contained within training-fold pipelines. The main configuration combined SMOTE with balanced class weights; three additional imbalance strategies tested sensitivity.

Selected-model measure	**Mean ± SD across 100 evaluations**
ROC-AUC	0.999 ± 0.001
Non-buyer average precision (reported as PR-AUC)	0.783 ± 0.184
Non-buyer recall	0.947 ± 0.135
Non-buyer precision	0.286 ± 0.103
MCC	0.512 ± 0.103

Logistic Regression led on PR-AUC, MCC and recall within every tested imbalance configuration. In the separate five-repeat out-of-fold analysis, a non-buyer score threshold of 0.50 caught an average 12.6 of 13 non-buyers while flagging 45.6 sessions per full pass. Approximately 33 flagged sessions were buyers. Increasing the threshold to 0.80 reduced alerts to 27.8 and retained 11.4 detected non-buyers.

Cart activity and previous purchases were recurring predictive signals; session duration and bounce rate added information. PCA described feature structure rather than predictive performance: its first three components explained 33.1% of feature variance and showed substantial class overlap.

The main deliverable is a reusable Logistic Regression pipeline, not a production-validated service. The data are simulated, minority evidence comes from only 13 unique records, prediction-time feature availability has not been established,

and external validation is absent. The recommended next step is prospective validation on representative real data before testing low-cost interventions.

2. Business Context, Scope and Research Questions

2.1 Illustrative Organizational Context

The project represents an illustrative retailer seeking to understand purchase-related session patterns and prioritize potentially non-converting sessions. It is not a case study of a named business, and the simulated class distribution should not be presented as typical of the e-commerce industry.

Potential stakeholders include analytics, marketing, customer support and product teams. Their decisions differ: support may prioritize recall, while a costly incentive program may require higher precision and demonstrable incremental margin. Predicting non-purchase is not the same as identifying customers whose behaviour can be changed by an intervention.

2.2 Business Question and Decision Boundaries

How effectively can the available session features rank and identify rare non-buyers, and what alert-volume versus missed-case trade-off does the fitted model offer under leakage-aware validation?

Current scope: retrospective classification using the available record-level features. Real-time abandonment prevention is a proposed future use, not an established capability. Before making such claims, define a scoring checkpoint and ensure that cart size, session duration, bounce measures and other predictors are observable before the purchase outcome and intervention.

The workflow therefore can:

- identify behavioural patterns associated with purchase outcomes;
- compare the ability of multiple models to rank and detect non-buyers;
- produce more informative performance estimates than accuracy;
- reveal how threshold choices affect missed non-buyers and false alerts; and
- provide a reusable modelling pipeline for future evaluation.

2.3 Objectives

Primary objective: Develop and evaluate a leakage-aware classification workflow for purchase prediction, with emphasizing the detection and ranking of the minority non-buyer class.

Supporting objectives

1. Audit data completeness, duplicates, outcome coding and class balance; describe buyer/non-buyer profiles.
2. Build a fold-isolated preprocessing and imbalance-handling workflow
3. Compare linear, single-tree and ensemble classifiers using minority-sensitive metrics and split-level variability.
4. Determine how thresholds change recall, precision, false alarms and alert volume.
5. Test whether model selection depends on SMOTE, categorical-aware resampling or class weighting.
6. Explain influential predictors without making causal or demographic-targeting claims.
7. Explore feature structure using real-row PCA and save a reproducible fitted pipeline.

2.4. Research Questions

1. How well can session-level behavioural data predict purchase outcomes in this dataset?
2. How does the extreme imbalance between buyers and non-buyers affect model evaluation and interpretation?
3. Which model performs best for non-buyer detection?
4. How do imbalance strategies and thresholds change results?
5. Which session-level features contribute most to the fitted models' predictions?
6. What patterns of feature variance and class overlap are visible in PCA projections?

3. Dataset and Exploratory Assessment

3.1 Source, Target and Unit of Analysis

The project uses a simulated dataset, as described by its publisher, containing demographic, device, engagement, and behavioural variables together with a binary purchase outcome. Findings should therefore not be generalized to live customer populations without external validation.

Attribute	Description

Platform	Kaggle
Dataset	E-Commerce Behavior Dataset (8000 Users)
File	`ecommerce_user_behavior_8000.csv`
Original size	8,000 records, 14 columns
Source	E-Commerce Behavior Dataset(8000 Users)

The project treats each record as a session-level analytical observation. Without source event histories, independence across repeated visits or users cannot be verified.

Field/ population	**Definition**
`purchase = 0`	Non-buyer; event of interest for precision, recall and average precision
`purchase = 1`	Buyer; majority class
`user_id`	Excluded from predictors; removed before exact-duplicate detection
Input/ transformed schema	12 raw predictors; 13 model features after one-hot encoding

Feature	**Type**	**Description**
`age`	Continuous	User age
`time_on_site`	Continuous	Time spent on the site in the session (minutes)
`pages_viewed`	Count, processed numerically	Number of pages viewed
`previous_purhases`	Count, processed numerically	Number of earlier purchases
`cart_items`	Count, processed numerically	Number of items added to the cart
`avg_session_time`	Continuous	Average session duration
`bounce_rate`	Continuous	Bounce-rate (percentage)
`discount_seen`	Binary	Whether a discount was shown

`ad_clicked`	Binary	Whether an advertisement was clicked
`returning_user`	Binary	Whether the user is returning
`gender`	Categorical	User gender
`device_type`	Categorical	Device used during the session

Time values and bounce-rate values follow the supplied data descriptions (minutes and percentage scale). Their exact business definitions, aggregation windows and availability at scoring time must be verified before operational use.

3.2 Data Quality and Cleaning Decisions

Data-quality item	**Result**
Raw records	8,000
Raw columns	14
Missing values in each original column	160 (2.0%)
Duplicate rows removed	0
Records removed because `purchase` was missing	160
Final modelling sessions	7,840
Buyers (`purchase = 1`)	7,827
Non-buyers (`purchase = 0`)	13
Non-buyer prevalence	0.17%

Every original column has 160 missing entries. Equal missingness counts do not establish that missingness occurs in the same rows, is random, or is independent of outcome. Unknown purchase labels were removed, not imputed. Missing predictors remain eligible for fold-specific imputation. No exact duplicates were found after dropping the ID.

IQR screening repoorted one potential outlier in `avg_session_time` and none in the other numeric columns. It was retained: statistical fence exceedance does not prove error, and deleting scarce minority records would be especially consequential. The workflow exports a data-quality audit table.

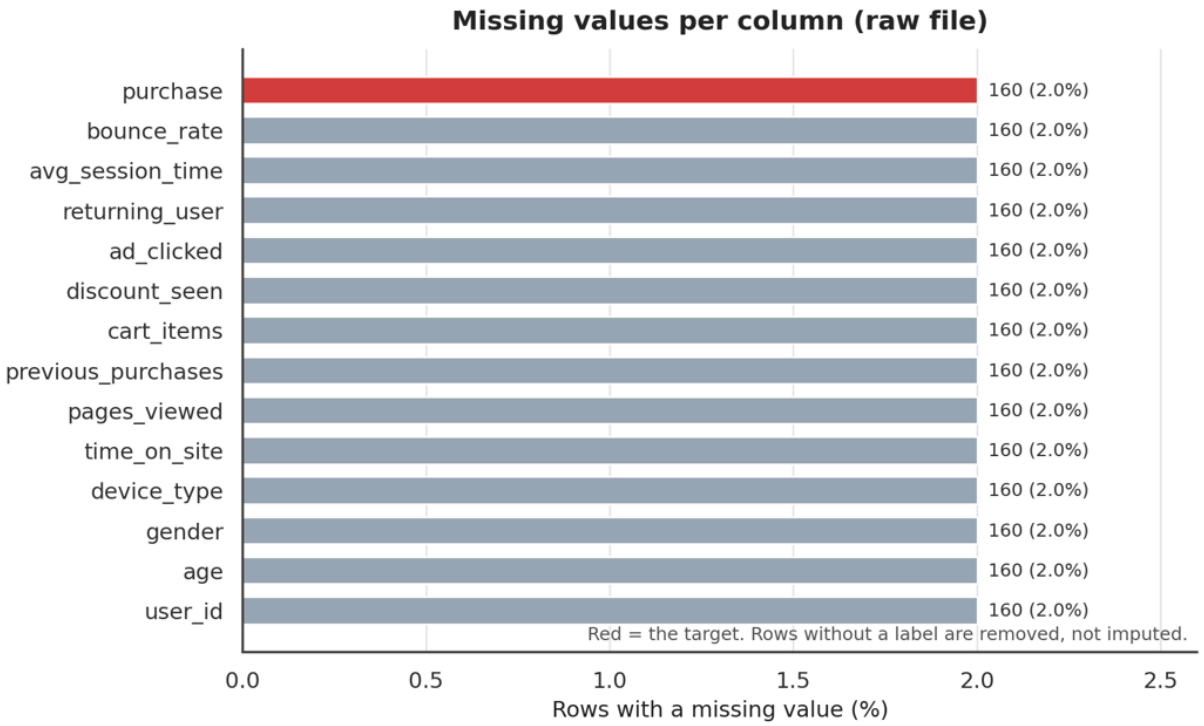


Figure 1. Missing values in the raw file. Unknown outcome labels are excluded; predictor missingness is imputed within training folds.

3.3 Extreme Imbalance and Descriptive Profiles

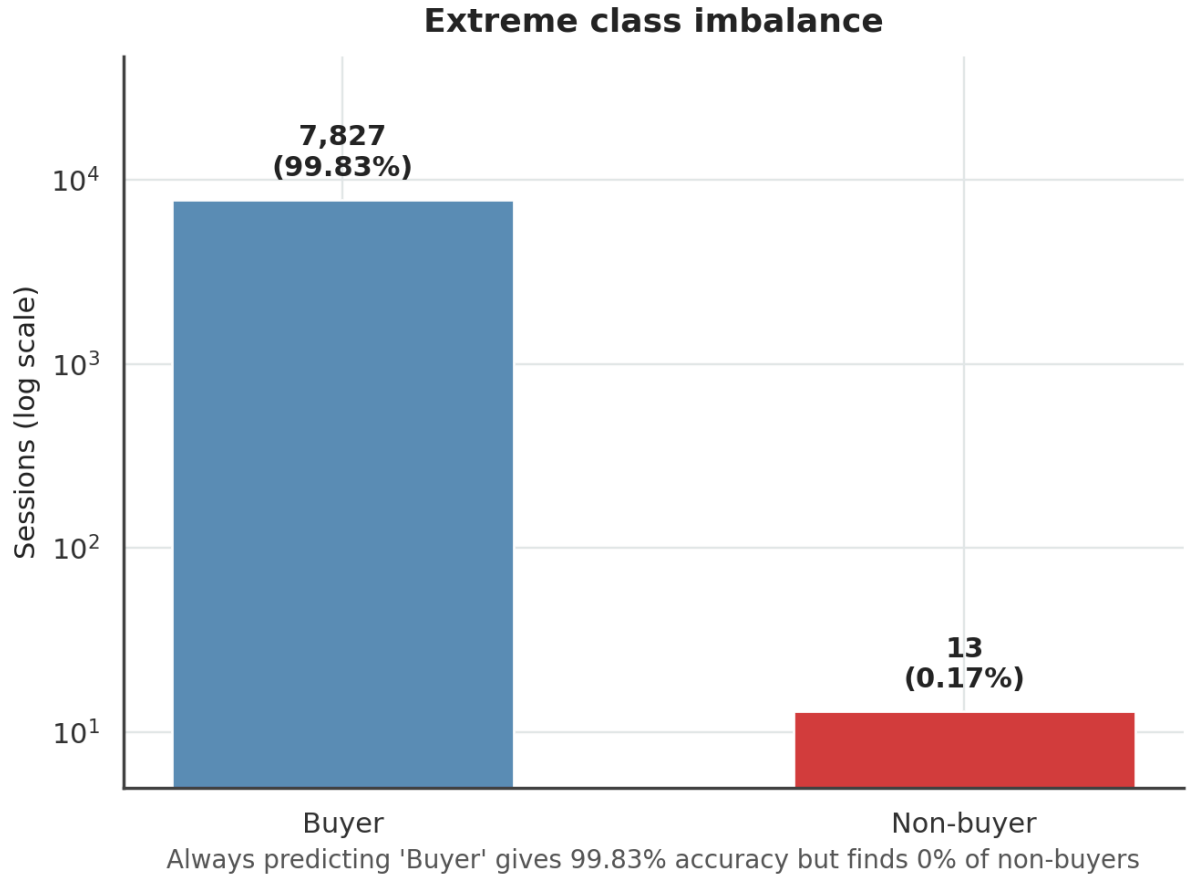


Figure 2. Buyer and non-buyer counts on a logarithmic scale. The labelled modelling population contains only 13 non-buyers.

Feature median	Buyers	Non-buyers
time_on_site	15.7	2.9
avg_session_time	14.3	2.8
pages_viewed	10.0	5.0
previous_purchases	7.0	1.0
cart_items	5.0	0.0
bounce_rate	49.4	89.9
age	39.0	35.0

In these observed records, non-buyers have shorter session times, fewer pages, fewer previous purchases, smaller carts and higher bounce rates. Age differs less. These summaries are descriptive: 13 minority observations cannot establish generalizable group effects or causal relationships.

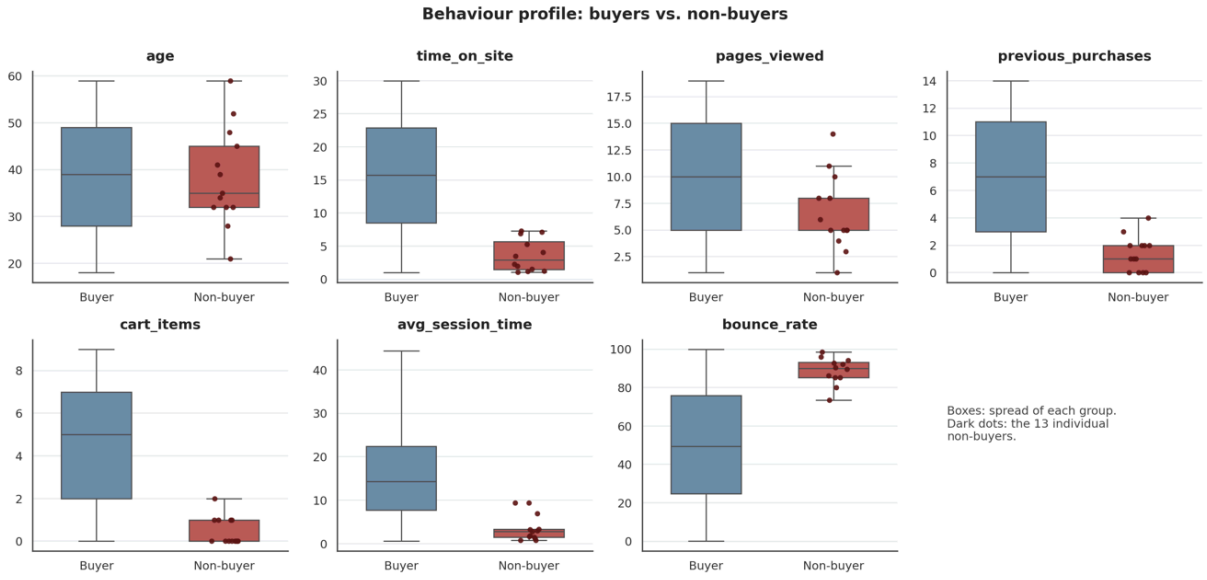


Figure 3. Numeric feature distributions by outcome, with individual non-buyer observations overlaid.

4. Methodology and Validation

4.1 Workflow Overview

The analysis follows a reproducible sequence: load and audit → drop identifier and exact duplicates → exclude missing labels → define X and Y → compare fold-fitted pipelines → generate repeated out-of-fold scores → test imbalance variants →

interpret fitted models → refit and serialize the selected pipeline → explore real-row PCA

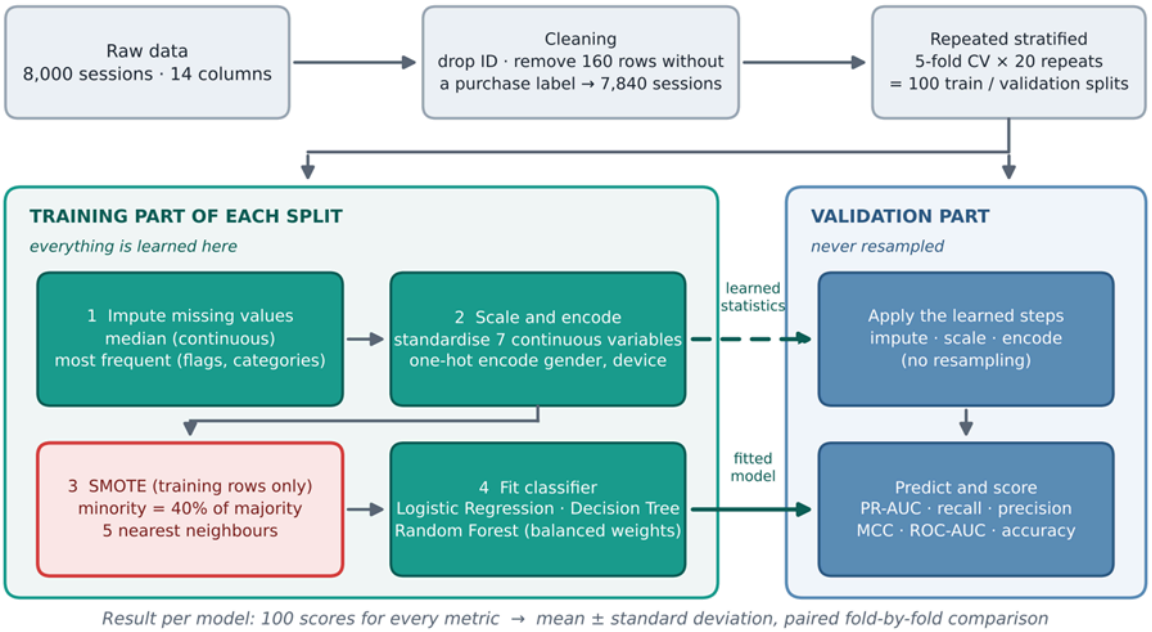


Figure 4. Training-fold workflow. Validation observations are transformed but never resampled.

The methodology is organized into the following stages:

Stage	Main activities	Purpose
1. Data preparation	Load data; inspect quality; drop identifiers and duplicates; remove rows with missing target	A consistent analysis set
2. Exploratory analysis	Examine missing values, class distribution, behavioral profiles, and potential outliers	Understand the data and its limits
3. Preprocessing	Impute missing values, standardize numerical variables; one-hot encode categorical variables	Prepare predictors for machine-learning algorithms
4. Imbalance handling	Apply SMOTE plus balanced class weights in the main configuration	Improve the treatment of the rare No Buy class
5. Model development	Train Logistic Regression, Decision Tree, and Random Forest	Compare linear and tree-based classification approaches
6. Model validation	Apply repeated stratified 5-fold cross-validation, 100 splits	Assess predictive performance and its variability

7. Threshold analysis	Generate out-of-fold probabilities, precision-recall curves, threshold comparisons, and average confusion matrix	Examine the recall/ alert-volume trade-off
8. Sensitivity analysis	Compare SMOTE, SMOTENC, and class weighting configurations	Test dependence on imbalance-handling strategies
9. Model interpretation	Analyze Logistic Regression coefficients, tree diagram and tree-based feature importances	Identify influential predictors and assess interpretation stability
10. Final model construction	Refit the selected Logistic Regression pipeline on all data and save the fitted pipeline	Produce a reusable model artifact
11. Exploratory PCA	Project real, unresampled sessions into principal-component space	Explore the structure of the dataset and possible behavioral patterns

4.2 Leakage Prevention

Each evaluated pipeline is **an imbalanced-learn Pipeline**: **ColumnTransformer → optional resampler → classifier. Medians, modes and scaling parameters are fitted using the training partition only**. Category vocabularies are predefined in the code, not discovered from validation data. Validation rows retain their original class balance. **SMOTE** is applied only to the preprocessed training partition. Validation data are never used to create synthetic examples. The full-data feature-name check fits a separate, discarded preprocessor; it is not reused in cross-validation. Full-data PCA is descriptive and is not part of predictive

```
steps = [("prep", make_preprocessor())]
```

```
return ColumnTransformer(
    [("continuous", continuous, CONTINUOUS),
     ("flags", flags, FLAGS),
     ("categorical", categorical, CATEGORICAL)],
    verbose_feature_names_out=False)
```

```
resampler = make_sampler(sampler)
if resampler is not None:
    steps.append(("resample", resampler))
```

```
steps.append(("clf", clf))
return ImbPipeline(steps)
```

4.3 Tools and Configuration

The project is implemented in Python using libraries for data manipulation, statistical visualization, machine learning, imbalanced classification, and model persistence.

Tool or Library	Role in the project
Python standard library (os, pathlib, textwrap)	Reads environment variables, manages file paths and output directories, and formats long chart labels
pandas	Loads and cleans the dataset, manages tabular data, calculates summaries, and exports analytical results
Numpy	Supports numerical computation, array manipulation, probability processing, and metric calculations
Matplotlib	Creates and exports the project's visualizations, including two-dimensional and three-dimensional charts
Seaborn	Produces statistical visualizations, including boxplots and comparative charts
scikit-learn	Provides preprocessing components, classification algorithms, evaluation metrics, cross-validation, PCA, and decision-tree visualization
imbalanced-learn	Provides SMOTE, SMOTENC, and a pipeline that integrates resampling into model training
joblib	Serializes the final fitted Logistic Regression pipeline for subsequent reuse

The principal tools are complemented by custom Python functions that standardize preprocessing, configure models, calculate minority-class metrics, cache computationally expensive results, generate out-of-fold predictions, and produce report-ready figures.

A fixed random state (`RANDOM_STATE = 42`) supports reproducibility of randomized steps. Reproducing results also requires the same data, compatible package versions, code, and configuration. Environment variables can change the input path and repeat counts.

Environment variables allow selected settings to be adjusted without modifying the source code. These include the dataset path (`ECOM_DATA`), the number of cross-validation repeats (`ECOM_REPEATS`), and the number of out-of-fold prediction repeats (`ECOM_OOF_REPEATS`).

```
DATA_PATH = Path(os.environ.get(
    "ECOM_DATA",
    r"C:\Users\abdel\E-commerce-Behavior-Analysis-Purchase-Prediction\Data\ecommerce_user_behavior_8000.csv"
    ))
```

```
N_SPLITS = 5
N_REPEATS = int(os.environ.get("ECOM_REPEATS", 20))
OOF_REPEATS = int(os.environ.get("ECOM_OOF_REPEATS", 5))
```

- `N_SPLITS = 5`: each fold uses approximately 80% of observations for training and 20% for validation.
- `N_REPEATS = 20`: the five-fold process is repeated 20 times, giving 100 model evaluations per model and configuration.
- `OOF_REPEATS = 5`: five repeated out-of-fold passes are used for precision-recall curves, threshold analysis, and the averaged confusion matrix.

```
RUN_ABLATIONS = True
USE_CACHE = True
SHOW_PLOTS = False
```

- `RUN_ABLATIONS = True`: enables comparisons of SMOTE, SMOTENC, and class weighting variants. Disabling it runs only the main configuration.
- `USE_CACHE = True`: reuses cached computations. Delete `cv_cache/` after changing code, data, or settings that affect cached results.
- `SHOW_PLOTS = False` saves figures without opening interactive windows. Set it to `True` to display them as well.

Output directories are created to organize the results:

- `figures/`: exported visualizations.
- `results/`: CSV result tables and the saved final model.
- `cv_cache/`: cached computational results used to avoid unnecessary repeated calculations.

```
FIG_DIR, OUT_DIR, CACHE_DIR = Path("figures"),
Path("results"), Path("cv_cache")
```

The `save_fig` helper exports each figure at report-quality resolution, optionally displays it, and closes it to release memory. The figure is saved before an optional display call.

```
def save_fig(fig, name):
    path = FIG_DIR / f"{name}.png"
    fig.savefig(path, bbox_inches="tight", facecolor="white")
    print(f"  saved {path}")
    if SHOW_PLOTS:
        plt.show()
    plt.close(fig)
```

Caching reduces run time, but not every code change is necessarily captured by the cache key. Clear `cv_cache/` whenever the modelling logic, features, data, scoring, or relevant configuration changes.

```
def cached(name, compute):
    path = CACHE_DIR / f"{name}.pkl"
    if USE_CACHE and path.exists():
        print(f"  [cache] loaded {name}")
        return pd.read_pickle(path)
    out = compute()
    if USE_CACHE:
        CACHE_DIR.mkdir(exist_ok=True)
        pd.to_pickle(out, path)
    return out
```

4.4 Data Preparation and EDA

4.4.1 Data loading and inspection

```
raw = pd.read_csv(DATA_PATH)
```

```
print(f"Raw shape: {raw.shape}")
print(raw.isna().sum().to_string())
fig_missing(raw)
```

The dataset is loaded from a CSV file into a pandas DataFrame. Its dimensions, column structure, and missing-value counts are examined before model development.

4.4.2 Identifier removal

```
df = raw.drop(columns=["user_id"])
```

The `user_id` column is removed before duplicate detection so distinct IDs do not conceal otherwise duplicated behavioural rows.

4.4.3 Duplicate removal

```
n_start = len(df)
df = df.drop_duplicates().reset_index(drop=True)
n_dup = n_start - len(df)
```

Duplicate rows are subsequently removed, the DataFrame index is reset to maintain consistent row indexing throughout the workflow. The number of removed duplicates is recorded for data-quality reporting.

4.4.4 Missing target values removal

```
n_mid = len(df)
df = df.dropna(subset=[TARGET]).reset_index(drop=True)
n_no_target = n_mid - len(df)
```

Rows with missing purchase outcomes are excluded. Missing predictor values are retained for fold-specific imputation in the pipeline.

4.4.5 Outlier Screening

```
outliers = pd.Series({
    c: int(((X[c] < X[c].quantile(.25) - 1.5 *
(X[c].quantile(.75) - X[c].quantile(.25))) |
            (X[c] > X[c].quantile(.75) + 1.5 *
(X[c].quantile(.75) - X[c].quantile(.25)))).sum())
    for c in CONTINUOUS
}, name="iqr_outliers")
print("IQR outliers (reported only, rows kept):")
print(outliers.to_string())
```

Potential outliers in continuous variables are flagged using the interquartile range (IQR), defined as Q3 - Q1. Values below Q1 - 1.5 × IQR or above Q3 + 1.5 × IQR are counted as potential outliers, but not removed.

This decision preserves potentially meaningful behavioral extremes and avoids discarding records when the minority class is already extremely small. IQR-based flags are treated as descriptive data-quality indicators, not proof that an observation is erroneous.

4.4.6 Data Quality Documentation

```
pd.DataFrame({
    "item": ["raw rows", "duplicate rows removed", "rows removed (missing target)",
             "rows used", "buyers", "non-buyers"],
    "value": [len(raw), n_dup, n_no_target, len(df), int((y == 1).sum()),
              int((y == 0).sum())]
}).to_csv(OUT_DIR / "data_quality_summary.csv", index=False)
```

A data-quality summary is exported to `results/data_quality_summary.csv`. It records key counts, including the original number of rows, duplicates removed, records excluded because of missing target values, final analytical row count, and buyer/non-buyer counts.

4.5 Preprocessing and Feature Schema

```
CONTINUOUS = ["age", "time_on_site", "pages_viewed",
              "previous_purchases", "cart_items",
              "avg_session_time", "bounce_rate"]

FLAGS = ["discount_seen", "ad_clicked", "returning_user"]

CATEGORICAL = ["gender", "device_type"]
GENDER_LEVELS = ["Female", "Male"]
DEVICE_LEVELS = ["Desktop", "Mobile", "Tablet"]
```

Feature group	Variables	Fold-specific preprocessing
Continuous numerical	`age, time_on_site, pages_viewed, previous_purchases, cart_items, avg_session_time, bounce_rate`	Median imputation; standardization
Binary flags	`discount_seen, ad_clicked, returning_user`	Most-frequent imputation; no model-stage scaling
Categorical	`gender, device_type`	Most-frequent imputation; fixed-category one-hot encoding; drop first; ignore unknown categories

4.5.1 Continuous variables

```
continuous = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler())
])
```

For numeric features, missing values are replaced with the median calculated from the training fold (median imputation is robust to extreme values) then standardized with `StandardScaler`: $z = (x - \mu) / \sigma$, where (x) represents the observed value, and μ and σ are the mean and standard deviation estimated from the training data.

Standardization is particularly relevant for Logistic Regression, optimized using numerical procedures sensitive to feature scales, SMOTE's nearest-neighbor distances on a common scale and the PCA considered variance structure.

4.5.2 Binary flags

```
flags = SimpleImputer(strategy="most_frequent")
```

For binary flags, missing values are replaced with the most frequent value observed in the training fold, not standardized to preserve their original indicator interpretation.

4.5.3 Categorical variables

```
categorical = Pipeline([
    ("impute", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(
        categories=[GENDER_LEVELS, DEVICE_LEVELS],
        drop="first",
        handle_unknown="ignore",
        sparse_output=False
    ))
])
```

Missing values in `gender` and `device_type` are imputed using the most frequent category in the training fold. `OneHotEncoder()`then converts them into numerical indicator columns using predefined category order (`*LEVELS`) and `drop="first"` to maintain a consistent feature structure and avoid redundant columns in Logistic Regression. Unknown categories are ignored rather than raising an error (`handle_unknown="ignore"`).

`Female` and Desktop are the reference categories.The encoded variables are interpreted as follows:

- `gender_Male = 1` indicates Male; `0` indicates Female.
- `device_type_Mobile = 1` indicates Mobile; `0` indicates not Mobile.
- `device_type_Tablet = 1` indicates Tablet; `0` indicates not Tablet.
- A session with both device indicators equal to zero is a Desktop session.

4.5.4 Feature-schema consistency

```
FEATURE_NAMES = CONTINUOUS + FLAGS + ["gender_Male",
"device_type_Mobile", "device_type_Tablet"]
```

Transformed feature names and the expected 13-column schema and order matching is checked avoiding feature-name misalignment in coefficient tables, tree diagrams, and importance plots. The output has:

1. Seven numeric columns,
2. Three binary flags,
3. Three indicators: `gender_Male`, `device_type_Mobile`, `device_type_Tablet`

```
CAT_IDX = list(range(len(CONTINUOUS), len(FEATURE_NAMES)))
```

In the 13-column transformed matrix, the six categorical/binary indices for SMOTENC are [7, 8, 9, 10, 11, 12].

4.5.5 Imbalance Treatment

```
SMOTE_RATIO = 0.4
K_NEIGHBORS = 5
```

```
if kind == "smote":
    return SMOTE(
        sampling_strategy=SMOTE_RATIO,
        k_neighbors=K_NEIGHBORS,
        random_state=RANDOM_STATE
    )
```

```
def make_models(class_weight=True):
    cw = "balanced" if class_weight else None
```

Tag	Configuration	Class weighting	Resampling method
main	SMOTE + class weight	Enabled	Standard SMOTE
`smote_only`	SMOTE only	Disabled	Standard SMOTE
`smotenc`	SMOTENC + class weight	Enabled	SMOTENC
`weight_only`	Class weight only	Enabled	None

SMOTE (Synthetic Minority Over-sampling Technique) creates synthetic non-buyers by interpolating between real non-buyers and their nearest non-buyer neighbours. It uses `SMOTE_RATIO=0.4` and `K_NEIGHBORS=5` raising minority count to 40% of majority count after oversampling in each training part.Training folds have 10 or 11 original non-buyers; the code verifies that the minimum exceeds k_neighbors.

Balanced class weights are calculated by the classifier from the labels it receives: weight(c) = N / (K × Nc). It raises the penalty for mistakes on the rare class without changing the data.

4.6 Model Configurations and Validation Design

4.6.1 Model Configurations

```python
def make_models(class_weight=True):
    cw = "balanced" if class_weight else None
    return {
        "Logistic Regression": LogisticRegression(
            max_iter=1000,
            class_weight=cw,
            random_state=RANDOM_STATE
        ),
        "Decision Tree": DecisionTreeClassifier(
            max_depth=5,
            class_weight=cw,
            random_state=RANDOM_STATE
        ),
        "Random Forest": RandomForestClassifier(
            n_estimators=200,
            max_depth=10,
            class_weight=cw,
            random_state=RANDOM_STATE,
            n_jobs=1
        )}
```

Three classification algorithms are evaluated under the main configuration:

Model	Main settings	Role
Logistic Regression	`max_iter=1000`, balanced class weights	Linear, probabilistic and interpretable baseline

Decision Tree	`max_depth=5`, balanced class weights	Simple non-linear rules; sensitive to few minority rows
Random Forest	200 trees, `max_depth=10`, `n_jobs=1`, balanced class weights	Ensemble that captures non-linearity and interactions

All classifiers use the same preprocessing pipeline, main imbalance strategy, and cross-validation splits. Parallelism is limited within the forest to avoid nested parallelism execution during cross-validation.

4.6.2 Cross-Validation Design

```
cv = RepeatedStratifiedKFold(
    n_splits=N_SPLITS,
    n_repeats=N_REPEATS,
    random_state=RANDOM_STATE )
min_minority = min(
    int((y.iloc[tr] == 0).sum()) for tr, _ in cv.split(X, y))
print(f"Smallest number of non-buyers in a training fold: {min_minority}")
assert min_minority > K_NEIGHBORS, "Too few minority rows for SMOTE: lower K_NEIGHBORS"
fingerprint = int(pd.util.hash_pandas_object(pd.concat([X, y], axis=1), index=False).sum() % 10**9)
```

Models are evaluated with `RepeatedStratifiedKFold` with the following configuration:

Parameter	**Setting**	**Explanation**
Number of folds	5	Each evaluation uses approximately 80% of the observations for training and 20% for validation
Number of repeats	20	Repeats the five-fold process using different shuffled splits
Total evaluations per model and variant	100	Five folds multiplied by 20 repeats
Random state	42	Supports reproducibility of the split assignments

Stratification preserves class proportions as closely as possible. Each validation fold has 1,568 rows, including two or three non-buyers. All models and variants use matched splits. Hyperparameters are fixed rather than optimized through a comprehensive search; results concern these configurations, not the algorithms' theoretical limits.

Mean ± SD uses NumPy's default population standard deviation (ddof=0) across the 100 scores. Repeated folds share observations, so SD is descriptive variability, not a confidence interval. Repetition reuses the same 13 unique non-buyers; it does not

create additional independent minority evidence. Matched-fold win rates are descriptive rather than significance tests.

```
def run_cv(X, y, cv, tag, class_weight, sampler, fingerprint):
    """Cross-validate the three models;
    returns {model: {metric: array of fold scores}}."""
    results = {}
    for name, clf in make_models(class_weight).items():
        def compute(clf=clf):
            res = cross_validate(
                make_pipe(clf, sampler),
                X,
                y,
                cv=cv,
                scoring=SCORING,
                n_jobs=-1,
                error_score="raise"
            )
            return {m: np.asarray(res[f"test_{m}"]) for m in SCORING}
        key = f"cv_{tag}_{name.replace(' ', '_')}_{N_REPEATS}x{N_SPLITS}_{fingerprint}"
        results[name] = cached(key, compute)
    return results
```

4.7 Evaluation Metrics and Conventions

4.7.1 Evaluation Metrics

```
SCORING = {
    "roc_auc": "roc_auc",
    "pr_auc": pr_auc_minority,
    "mcc": make_scorer(matthews_corrcoef),
    "f1_weighted": "f1_weighted",
    "accuracy": "accuracy",
    "recall_minority": make_scorer(recall_score,
    pos_label=0, zero_division=0),
    "precision_minority": make_scorer(precision_score,
    pos_label=0, zero_division=0),
}
```

The evaluation includes seven metrics selected and calculated in each cross-validation fold to provide complementary views of classification performance:

Metric	Interpretation	Implementation
PR-AUC/	`average_precision_score`	Primary ranking measure

average precision	on `y == 0` using P(class 0). This is AP, not trapezoidal integration.	sensitive to rare-class rankings
Non-buyer recall	TP / (TP + FN), with class 0 treated as the event of interest.	How many non-buyers are found
Non-buyer precision	TP / (TP + FP). Undefined fold precision is set to 0 by `zero_division=0.`	Share of flagged sessions that are real non-buyers Measures the reliability of alerts
MCC	Correlation between predicted and actual classes using all four confusion-matrix outcomes	a complementary threshold-based measure: balanced single score under class imbalance
ROC-AUC	Code uses the standard class-1 scorer. Reversing both labels and scores gives the equivalent non-buyer ranking interpretation.	Context: stays high despite rare-class poor handling
Weighted F1	F1 averaged by class size	Context measure dominated by the buyer majority
Accuracy	Share of all sessions classified correctly	Reported but not used for selection

Precision and recall use class `0` (non-buyer) as the positive label. ROC-AUC and PR-AUC use predicted probabilities; threshold-based metrics use the default 0.5 cutoff unless otherwise stated. PR-AUC is implemented as average precision. Its random-ranking baseline is approximately the non-buyer prevalence, around 0.17%.

```python
def pr_auc_minority(estimator, X_te, y_te):
    """PR-AUC for the minority class (No Buy = 0),
    read from that class's probability."""
    col = list(estimator.classes_).index(0)
    return average_precision_score(
        (y_te == 0).astype(int),
        estimator.predict_proba(X_te)[:, col]
    )
```

4.7.2 Aggregation and Reporting

```python
return {m: np.asarray(res[f"test_{m}"]) for m in SCORING}
```

```
label, cw, sampler = VARIANTS["main"]
print(f"\n=== Main evaluation: {label} ({N_SPLITS}x{N_REPEATS} CV) ===")
results = {"main": run_cv(X, y, cv, "main", cw, sampler, fingerprint)}
```

For each model, the evaluation records the metric values obtained across the 100 validation evaluations such as `test_roc_auc, test_pr_auc,test_mcc`.

The results are summarized using the mean and standard deviation for each metric.

```
def summarize(results):
    return pd.DataFrame(
        {n: {m: f"{r[m].mean():.3f} ± {r[m].std():.3f}" for m in SCORING}
         for n, r in results.items()}).T
```

The mean summarizes observed performance over the evaluated splits; the standard deviation describes split-to-split variability. Because repeated cross-validation reuses the same observations, these values should not be independent-sample uncertainty intervals.

```
def means(results):
    return pd.DataFrame(
        {n: {m: r[m].mean() for m in SCORING}
         for n, r in results.items()}).T
```

The workflow exports both a formatted summary table containing mean ± standard deviation and a numerical table containing the corresponding means. The results are used to generate comparative visualizations and support model-selection decisions.

```
summary = summarize(results["main"])
print(summary.to_string())
summary.to_csv(OUT_DIR / "cv_results_summary.csv")
means(results["main"]).to_csv(OUT_DIR / "cv_results_means.csv")
```

4.7.3 Paired Model Comparisons

```python
def paired_comparison(results, reference="Logistic Regression",
                      metrics=("pr_auc", "recall_minority", "mcc", "precision_minority")):
    rows = []
    for other in results:
        if other == reference:
            continue
        for m in metrics:
            diff = results[reference][m] - results[other][m]
            rows.append({
                "metric": m,
                "vs": other,
                "mean_difference": diff.mean(),
                "reference_better_pct": (diff > 0).mean() * 100,
                "tied_pct": (diff == 0).mean() * 100
            })
    return pd.DataFrame(rows)
```

```python
pairs = paired_comparison(results["main"])
print(pairs.round(3).to_string(index=False))
pairs.to_csv(OUT_DIR / "paired_comparison.csv", index=False)
fig_cv_comparison(results["main"])
```

Because the three models are evaluated on the same cross-validation splits, their scores can be compared fold by fold.

Logistic Regression is used as the reference model for descriptive comparisons against the Decision Tree and Random Forest. For selected metrics, the analysis calculates:

- The average difference in performance between the reference model and the competing model.
- The percentage of matched folds in which Logistic Regression performs better.
- The percentage of folds in which the models produce tied scores.

Paired fold comparisons are descriptive. Since repeated cross-validation reuses observations and splits are dependent, better-in-fold percentages are not formal hypothesis tests.

4.7.4 Model-comparison Visualization

The cross-validation comparison figure focuses on four metrics:

- Minority-class PR-AUC.
- MCC.
- Minority-class recall.
- Minority-class precision.

The figure displays the distribution of fold-level results, including quartiles, individual observations, and mean values. This reveals whether a model's average performance is supported by relatively consistent results or is driven by a subset of particularly favorable splits.

Variability is especially important in this project because the minority class is represented by only 13 original observations.)

4.8 Out-of Fold and Threshold Analysis

```
print("\n=== Out-of-fold predictions ===")
oof = oof_scores(X, y, fingerprint)
table = fig_operating_points(oof, y)
table.to_csv(OUT_DIR / "threshold_analysis_logistic_regression.csv", index=False)
print(table.round(3).to_string(index=False))
```

```
def oof_scores(X, y, fingerprint):
    """Out-of-fold P(No Buy) for each model,
    OOF_REPEATS times (every row scored once per repeat)."""
    out = {}
    for name, clf in make_models(True).items():
        def compute(clf=clf):
            rows = []
            for r in range(OOF_REPEATS):
                skf = StratifiedKFold(
                    n_splits=N_SPLITS,
                    shuffle=True,
                    random_state=RANDOM_STATE + r)
                proba = cross_val_predict(
                    make_pipe(clf, "smote"),
                    X, y,
                    cv=skf,
                    method="predict_proba",
                    n_jobs=-1)
                rows.append(proba[:, 0])
            return np.vstack(rows)
        out[name] = cached(f"oof_{name.replace(' ', '_')}_{OOF_REPEATS}_{fingerprint}", compute)
    return out
```

O**ut-of-fold probabilities** are generated with five repeated stratified five-fold passes. Within each pass, every session receives a prediction from a model that did not train on it. These probabilities support pooled precision-recall curves, threshold analysis, and the averaged confusion matrix.

```python
def threshold_table(scores, y_min, thresholds):
    rows = []
    for t in thresholds:
        flagged = scores >= t
        tp = (flagged & (y_min == 1)).sum(axis=1)
        n_flag = flagged.sum(axis=1)

        rows.append({"threshold": t,
                     "recall": (tp / y_min.sum()).mean(),
                     "precision": np.where(n_flag > 0, tp /
                                           np.maximum(n_flag, 1), np.nan).mean(),
                     "flagged_per_run": n_flag.mean(),
                     "caught_per_run": tp.mean()
                    })
    return pd.DataFrame(rows)
```

For each threshold, the workflow calculates Recall, Precision, Flagged sessions per run, and the Non-buyers caught per run. The resulting table is exported to `results/threshold_analysis_logistic_regression.csv`.

4.9 Sensitivity Analysis

Because the conclusions could depend on how imbalance is handled, all three models are re-evaluated under four configurations on identical splits.

```python
VARIANTS = {
    "main":         ("SMOTE + class weight", True, "smote"),
    "smote_only":   ("SMOTE only", False, "smote"),
    "smotenc":      ("SMOTENC + class weight", True, "smotenc"),
    "weight_only": ("Class weight only (no resampling)", True, None),
}
```

This dictionary defines the four configurations compared in the sensitivity analysis

Standard SMOTE interpolates numerically to create fractional values in binary and one-hot-encoded columns.

SMOTENC treats the 6 binary and one-hot columns (positions 7 to 12 of the 13 transformed features) as categorical. The class-weight-only run isolates the effect of synthetic rows.

```
if kind == "smotenc":
    return SMOTENC(
        categorical_features=CAT_IDX,
        sampling_strategy=SMOTE_RATIO,
        k_neighbors=K_NEIGHBORS,
        random_state=RANDOM_STATE
    )
```

For each configuration, the script records cross-validation performance and calculates mean metrics for all three classifiers. The results are combined into `results/ablation_comparison.csv` and displayed in a comparative visualization.

```
if RUN_ABLATIONS:
    for tag in ["smote_only", "smotenc", "weight_only"]:
        label, cw, sampler = VARIANTS[tag]
        print(f"\n=== Sensitivity: {label} ===")
        results[tag] = run_cv(X, y, cv, tag, cw, sampler, fingerprint)
        summarize(results[tag]).to_csv(OUT_DIR / f"cv_results_{tag}.csv")
    variant_means = {tag: means(r) for tag, r in results.items()}
    key = ["pr_auc", "mcc", "recall_minority", "precision_minority"]
    comparison = pd.concat({
        VARIANTS[t][0]: m[key].round(3)
        for t, m in variant_means.items()
    }, axis=1)
    print(comparison.to_string())
    comparison.to_csv(OUT_DIR / "ablation_comparison.csv")
    fig_ablation(variant_means)
```

The sensitivity analysis compares the specified configurations on common cross-validation splits. It tests robustness within this workflow; it does not establish a universally best resampling method.

4.10 Model Interpretation

```
print("\n=== Interpretation ===")
lr_smote = fold_models(X, y, make_models(True)["Logistic Regression"], "smote", cv)
lr_plain = fold_models(X, y, make_models(True)["Logistic Regression"], None, cv)
coef_smote = np.array([m.coef_[0] for m in lr_smote])
coef_plain = np.array([m.coef_[0] for m in lr_plain])
fig_lr_coefficients(coef_smote, coef_plain)
```

- **Logistic Regression coefficients** are collected across training folds with and without SMOTE. Since `purchase = 1` denotes buyers, positive coefficients are associated with higher buyer log-odds and negative coefficients are predictive associations, not causal effects.
- **Tree-based impurity importances** are collected from 100 Decision Tree fits and five Random Forest fits. A separate Decision Tree fitted on all cleaned data is visualized to its first three levels. Importances are model-dependent and do not indicate effect direction or causality. The corresponding summary is exported as `results/tree_feature_importance.csv`.

```
dt_folds = fold_models(X, y, make_models(True)["Decision Tree"], "smote", cv)
rf_folds = fold_models(X, y, make_models(True)["Random Forest"], "smote",
    StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=RANDOM_STATE))
imp_dt = np.array([m.feature_importances_ for m in dt_folds])
imp_rf = np.array([m.feature_importances_ for m in rf_folds])
fig_importances(imp_dt, imp_rf)

pd.DataFrame({
    "feature": FEATURE_NAMES,
    "dt_importance_mean": imp_dt.mean(0),
    "dt_importance_std": imp_dt.std(0),
    "rf_importance_mean": imp_rf.mean(0),
    "rf_importance_std": imp_rf.std(0)
    }).sort_values(
        "rf_importance_mean",
        ascending=False
    ).to_csv(OUT_DIR / "tree_feature_importance.csv", index=False)
```

After model comparison, a Decision Tree is fitted using all cleaned observations under the selected SMOTE-based configuration. The first three levels of the fitted tree are visualized to present its principal split variables and decision structure. The visualization includes readable feature names, class labels, split thresholds, sample counts, and class-weighted values.

```
def fig_tree(tree_clf):
    fig, ax = plt.subplots(figsize=(22, 9))
    plot_tree(
        tree_clf, max_depth=3,
        feature_names=FEATURE_NAMES,
        class_names=["No Buy", "Buy"],
        filled=True, rounded=True,
        impurity=False, fontsize=10,
        precision=2, ax=ax
    )
```

The diagram is intended to illustrate the structure of a fitted tree, not to demonstrate superior predictive performance or provide an unbiased evaluation of its generalization.

4.11 Final Model

```
final_lr = make_pipe(make_models(True)["Logistic Regression"], "smote").fit(X, y)
joblib.dump(final_lr, OUT_DIR / "final_logistic_regression_pipeline.joblib")

coef_table = pd.DataFrame({
    "feature": FEATURE_NAMES,
    "final_fit": final_lr.named_steps["clf"].coef_[0],
    "fold_mean": coef_smote.mean(0), "fold_std": coef_smote.std(0),
    "no_smote_fold_mean": coef_plain.mean(0),
    "no_smote_fold_std": coef_plain.std(0)
}).sort_values("final_fit", ascending=False)
print(coef_table.round(3).to_string(index=False))
coef_table.to_csv(OUT_DIR / "lr_coefficients.csv", index=False)

final_dt = make_pipe(make_models(True)["Decision Tree"], "smote").fit(X, y)
fig_tree(final_dt.named_steps["clf"])
```

After model comparison, the selected Logistic Regression pipeline is fitted on all 7,840 labelled sessions and saved as `results/final_logistic_regression_pipeline.joblib`. This artifact is for reuse, not unbiased assessment; performance claims rely on cross-validation and out-of-fold estimates. Conceptually, the supported operations are:

- `pipeline.predict(new_data)`: returns predicted class labels.
- `pipeline.predict_proba(new_data)`: returns predicted class probabilities.

important limitation: The all-data fitted pipeline is a reusable final model artifact, not a source of unbiased performance estimates. Its fitting process has access to all cleaned observations, so its predictions on those same observations cannot be treated as independent validation results.

All claims about predictive performance and model comparison must therefore be based on the cross-validation and out-of-fold analyses, not on the final model's fit to the complete dataset.

4.12 Exploratory Principal Component Analysis

4.12.1 PCA preprocessing

```
Z = make_preprocessor().fit_transform(X)
Z_std = StandardScaler().fit_transform(Z)
```

PCA is performed on the real, unresampled sessions after preprocessing and standardization. This exploratory analysis describes the feature matrix; it does not use SMOTE-generated observations.

4.12.2 Explained Variance and Loadings

```
scores, ev, loadings, buyers, non_buyers = pca_view(Z_std, y)
print("Variance explained:", np.round(ev[:3] * 100, 1),
      "| first three:", round(ev[:3].sum() * 100, 1), "%")
print(loadings.abs().sort_values("PC1", ascending=False).round(3).to_string())

loadings.to_csv(OUT_DIR / "pca_loadings.csv")
fig_pca3d(scores, ev, buyers, non_buyers)
print("\nDone. Figures in figures/, tables and the final model in results/.")
```

The PCA analysis generates:

- The proportion of variance explained by each principal component.
- The cumulative explained variance.
- A feature-loading table for the first three components.
- A heatmap showing the component loadings.

Explained variance describes the share of variation in the standardized predictor matrix captured by each component. Loadings describe how transformed features contribute to a component. Component signs are arbitrary up to reversal, so loadings should be interpreted by relative magnitude and pattern, not as effects on purchase.

The PCA figures show explained variance, the first three component loadings, PC1–PC2 and PC1–PC3 projections, and a three-dimensional projection. To improve readability, up to 2,500 buyers are sampled for the scatterplots while all 13 non-buyers are retained.

4.12.3 PCA Limitations

PCA is an unsupervised exploratory technique. Visible separation between buyers and non-buyers may suggest structural differences in the predictor space, but does not establish reliable class separability or demonstrate predictive performance. Conversely, overlap in the first few components does not prove that the classes cannot be distinguished using the original features or nonlinear decision boundaries. The PCA figures are therefore interpreted as exploratory visual evidence rather than a substitute for supervised cross-validation.

4.13 Output Management and Reporting

The workflow produces a collection of figures, result tables, and a fitted model artifact. These outputs support reproducibility, interpretation, and the preparation of the analytical report.

4.13.1. Visualizations

Figure	Content
Figure 1	Missing-value distribution
Figure 2	Buyer/non-buyer class imbalance
Figure 3	Continuous behavioral profiles by class
Figure 4	Cross-validation model comparison
Figure 5	Precision-recall curves, threshold trade-offs, and averaged confusion matrix
Figure 6	Sensitivity analysis of imbalance-handling strategies
Figure 7	Logistic Regression coefficient comparison
Figure 8	First three levels of the fitted Decision Tree
Figure 9	Decision Tree and Random Forest feature importance
Figure 10	PCA overview, explained variance, loadings, and two-dimensional projections
Figure 11	Three-dimensional PCA projection

4.13.2. Result Files

The following outputs preserve the numerical results supporting the analysis:

File	Purpose
`data_quality_summary.csv`	Records key data-cleaning and class-distribution counts
`cv_results_summary.csv`	Stores formatted mean ± standard deviation cross-validation results
`cv_results_means.csv`	Stores numerical mean performance metrics

`paired_comparison.csv`	Records fold-matched descriptive comparisons with Logistic Regression
`threshold_analysis_logistic_regression.csv`	Stores threshold-level precision, recall, flagged-session, and caught-session results
`cv_results_smote_only.csv`	Stores results for the SMOTE-only configuration
`cv_results_smotenc.csv`	Stores results for the SMOTENC-plus-class-weighting configuration
`cv_results_weight_only.csv`	Stores results for the class-weighting-only configuration
`ablation_comparison.csv`	Combines the sensitivity-analysis results
`tree_feature_importance.csv`	Stores Decision Tree and Random Forest feature-importance summaries
`lr_coefficients.csv`	Stores final and cross-validation Logistic Regression coefficient summaries
`pca_loadings.csv`	Stores feature loadings for the selected principal components
`final_logistic_regression_pipeline.joblib`	Stores the final fitted Logistic Regression pipeline

Figure communicate distributions and model patterns; CSV files preserve the numeric results. The serialized all-data model is a reusable artifact and must not be mistaken for an independently validated model.

5. Results and Interpretations

5.1 Cross-validated Model Comparison

The table reports the mean and standard deviation across the 100 evaluations.

Model	ROC-AUC	PR-AUC	MCC	F1 weighted	Accuracy	Recall_ minority	Precision _minority
Logistic Regression	**0.999**± 0.001	**0.783**± 0.184	**0.512**± 0.103	0.997± 0.001	0.996± 0.001	**0.947**± 0.135	**0.286**± 0.103
Decision Tree	0.752± 0.158	0.115± 0.104	0.230± 0.177	0.997± 0.001	0.996± 0.002	0.367± 0.294	0.163± 0.170

Random Forest	0.991± 0.026	0.498± 0.248	0.314± 0.317	0.998± 0.001	0.999± 0.001	0.242± 0.258	0.432± 0.452

Logistic Regression is best on **ROC-AUC (0.999 ± 0.001) and PR-AUC (0.783 ± 0.184)**, indicating highly consistent ranking performance.

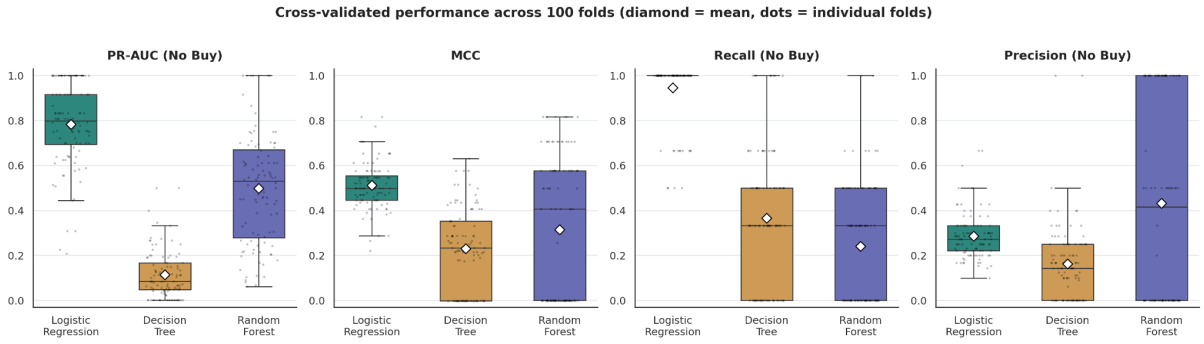
It achieved the **highest non-buyer recall of 0.947 ± 0.135 at** the default 0.5 threshold, corresponding to identifying approximately **94.7% of actual non-buyers** and **missing only around 5.3% on average**. Its **MCC of 0.512 ± 0.103 was also the highest**, indicating the best balance across correct and incorrect classifications among the three models. Its relatively low variability across folds further supports its reliability.

Its **non-buyer precision of 0.286 ± 0.103** means approximately **28.6%** of sessions flagged as non-buyers were truly non-buyers. This is moderate in absolute terms, but substantially above the dataset's non-buyer prevalence of about 0.17%. The model therefore enriches the flagged group by roughly **170 times** relative to selecting sessions at random.

The Decision Tree is the weakest and least stable model, as expected when a single tree is grown from about 10 minority rows. It had the lowest minority-class performance. **It missed approximately 63.3% of non-buyers.**

Random Forest achieved the highest accuracy and the **highest average precision at 0.498 ± 0.248**. However, it finds only **24% of non-buyers** and **its precision varies enormously (0.432 ± 0.452)**. It behaves as a conservative model: it flagged fewer sessions, but failed to identify most actual non-buyers.

Accuracy and weighted F1 are almost identical for all three models (0.996 to 0.999) and are dominated by the buyer majority. Predicting every session as a buyer would yield approximately **99.83% accuracy** while detecting none of the non-buyers. Random Forest's slightly higher accuracy therefore does not make it the best model for the project's objective.



Paired comparison on identical splits

The three models reveal a clear difference between overall classification performance and performance on the rare class of interest.

Logistic Regression vs	**Metric**	**Mean difference**	**LR better**	**Tied**
Decision Tree	PR-AUC	+0.668	100%	0%
	Recall	+0.580	90%	10%
	MCC	+0.282	91%	1%
	Precision	+0.123	79%	2%
Random Forest	PR-AUC	+0.285	85%	5%
	Recall	+0.705	96%	4%
	MCC	+0.198	62%	0%
	Precision	-0.146	49%	1%

Fold-level paired comparisons further support Logistic Regression. It outperformed the Decision Tree on PR-AUC in every split and leads on recall by a wide margin.

Against Random Forest the picture is more nuanced: it wins PR-AUC in 85% of splits and recall in 96%, but only 62% on MCC, and Random Forest has the higher mean precision (difference −0.146, with Logistic Regression ahead in 49% of splits). The preference for Logistic Regression rests mainly on recall and ranking, not on a decisive win everywhere.

The paired comparison therefore reinforces the central trade-off: Logistic Regression detects more non-buyers, while Random Forest is more selective when making non-buyer predictions.

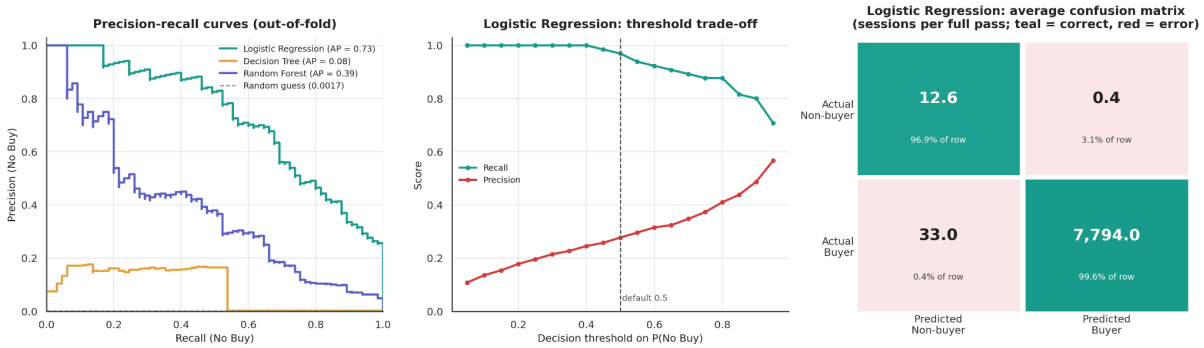
Criterion	**Logistic Regression**	**Decision Tree**	**Random Forest**	**Preferred model**
Broad ranking quality	**0.999** ROC-AUC	0.752 ROC-AUC	0.991 ROC-AUC	Logistic Regression
Non-buyer ranking quality	**0.783** PR-AUC	0.115 PR-AUC	0.498 PR-AUC	Logistic Regression
Non-buyer detection rate	**94.7%** recall	36.7% recall	24.2% recall	Logistic Regression
Reliability	High	Low	Low to	Logistic Regression

across folds			moderate	
Non-buyer precision	28.6%	16.3%	**43.2%**	Random Forest, but highly unstable
Balanced classification quallity	**0.512** MCC	0.230 MCC	0.314 MCC	Logistic Regression
Interpretability	High	High	Moderate	Logistic Regression / Decision Tree
Overall ranking ability	Best and most stable	moderate	very strong but less stable than LR	Logistic Regression

Overall, **Logistic Regression is the preferred model for this recall and ranking-focused objective**. It leads on **ROC-AUC, PR-AUC, MCC, and non-buyer recall** while showing **comparatively stable fold-level performance**. Random Forest has higher mean precision and accuracy but substantially lower recall; the Decision Tree is weaker and less stable.

5.2 Threshold and Operating-point Analysis

Precision-recall curves from the pooled out-of-fold probabilities, an operating table for Logistic Regression and the averaged confusion matrix at the default 0.5 threshold are shown in the Figure below



The Precision-recall curve compares ranking behaviour over thresholds. The threshold table reports the operational consequences of selected cutoffs in terms of recall, precision, and the number of sessions flagged.

5.2.1 Precision-Recall curves

Pooled out-of-fold average precision is approximately 0.73 for Logistic Regression, 0.39 for Random Forest, and 0.08 for Decision Tree. The model ranking agrees with

the fold-level results, although pooled OOF average precision is not directly interchangeable with the mean of fold-level PR-AUC values.

Model	Pooled out-of-fold precision
Logistic Regression	0.73
Random Forest	0.39
Decision Tree	0.08

The random-ranking reference is approximately the non-buyer prevalence (about 0.17%). Logistic Regression's pooled curve is substantially above this reference and the other models, indicating stronger prioritization of non-buyers in these out-of-fold predictions.

5.2.2 Threshold trade-off and average confusion matrix

The table shows how the model behaves as the threshold on P(No Buy) changes.

Threshold	Recall	Precision	Sessions flagged per pass	Non-buyers caught (of 13)
0.40	1.000	0.245	53.2	13.0
0.50	**0.969**	**0.277**	**45.6**	**12.6**
0.60	0.923	0.315	38.2	12.0
0.70	0.892	0.347	33.4	11.6
0.80	0.877	0.410	27.8	11.4
0.90	0.800	0.487	21.4	10.4
0.95	0.708	0.567	16.4	9.2

At threshold 0.50, the averaged out-of-fold confusion matrix contains approximately **12.6** true positives, 0.4 false negatives, 33.0 false positives, and 7,794.0 true negatives per full pass. About 28% of flagged sessions are non-buyers; the remainder are buyers, illustrating the cost of the model's high recall.

Actual class	Predicted non-buyer	Predicted buyer
Non-buyer	12.6 true positives	0.4 false negatives
Buyer	33.0 false positives	7,794.0 true negatives

A 0.40 threshold catches all 13 non-buyers on average while flagging about 53 sessions. At 0.95, the model flags about 16 sessions but catches approximately 9.2 non-buyers. These are averaged repeated out-of-fold counts, not observations from one fixed test set.

Therefore, the threshold should be chosen according to the relative costs of missing a non-buyer and contacting or intervening with a buyer. A lower threshold is appropriate when high recall is important and interventions are low-cost; a higher threshold may be appropriate when alert volume or intervention costs are constrained.

Interpreting the two recall estimates

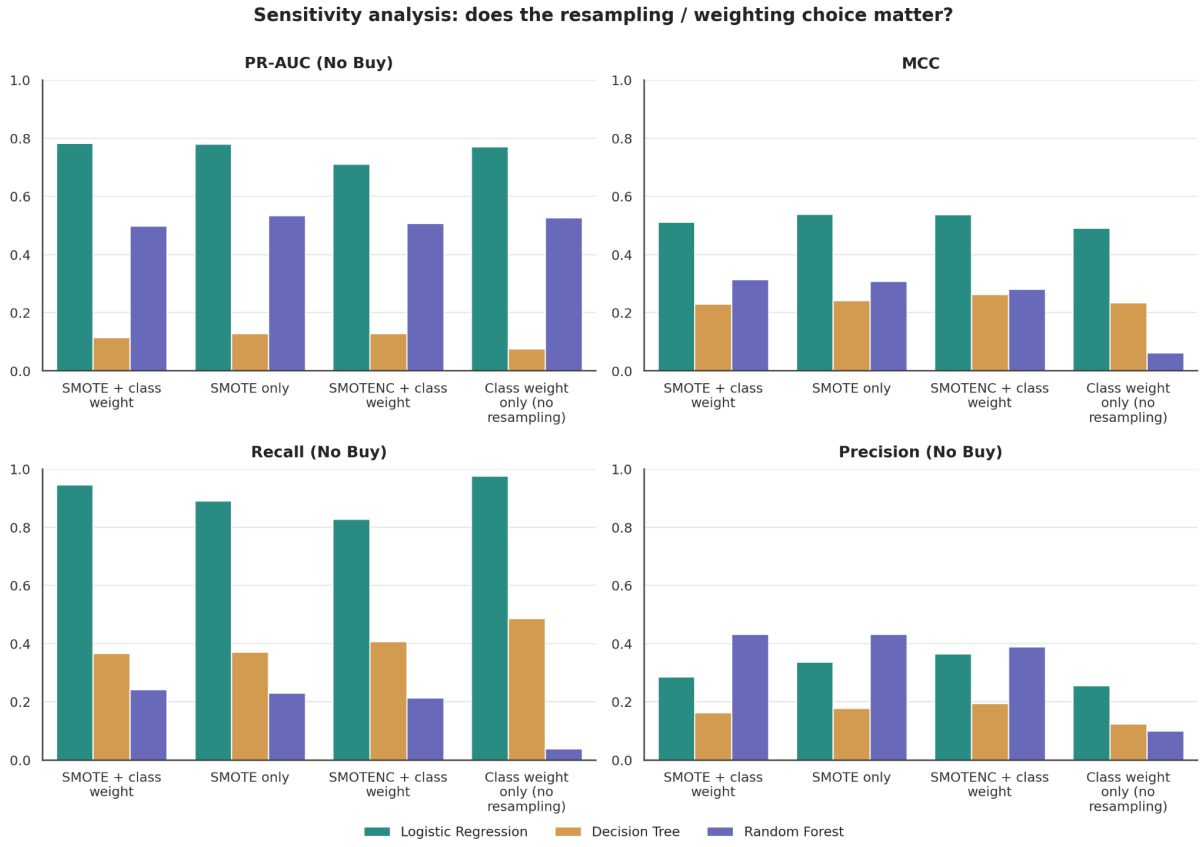
The main cross-validation table and the threshold table report related but differently aggregated estimates at a threshold of 0.50.
The table's 0.947 is the mean of 100 fold-level recall scores; each validation fold contains only a few non-buyers, so one error can materially change a fold's recall.
The threshold table's 0.969 is based on average caught counts over the five repeated out-of-fold passes (12.6 of 13 per pass). The estimates are not contradictory: they summarize different validation aggregations.

5.3 Sensitivity Analysis

The charts report mean performance for the three classifiers on the non-buyer class. They compare PR-AUC, MCC, Recall, and Precision.

Configuration	Model	PR-AUC	MCC	Recall	Precision
SMOTE + class weight (main)	Logistic Regression	0.783	0.512	0.947	0.286
	Decision Tree	0.115	0.230	0.367	0.163
	Random Forest	0.498	0.314	0.242	0.432
SMOTE only	Logistic Regression	0.780	0.539	0.892	0.336
	Decision Tree	0.128	0.242	0.372	0.178
	Random Forest	0.535	0.308	0.230	0.432
SMOTENC + class weight	Logistic Regression	0.711	0.538	0.828	0.365

	Decision Tree	0.129	0.263	0.407	0.194
	Random Forest	0.507	0.281	0.213	0.388
class weight only (no resampling)	Logistic Regression	0.771	0.491	0.977	0.255
	Decision Tree	0.075	0.235	0.487	0.124
	Random Forest	0.526	0.062	0.038	0.100



- **Logistic Regression has the highest** PR-AUC in every configuration, (0.711 to 0.783), so the model choice does not hinge on a particular resampling method.
- **SMOTE adds little for Logistic Regression.** PR-AUC is 0.783 with SMOTE and class weights, 0.780 with SMOTE alone and 0.771 with class weights alone, differences far inside the split-to-split spread of about ± 0.18. Its main effect is to trade recall for precision: **class weights alone give recall 0.977 and precision 0.255**, **SMOTE alone gives 0.892 and 0.336.**

- For Logistic Regression, class weighting alone produced the highest recall (0.977), but lower precision (0.255). It represents a more aggressive operating point.
- SMOTE alone gives the best Logistic Regression MCC (0.539), and SMOTENC plus class weights the highest precision (0.365), but the lowest PR-AUC (0.711).
- The Decision Tree remained weak across configurations, suggesting that changing the imbalance strategy did not resolve its limitations.
- **SMOTE matters for Random Forest.** Without resampling, its recall falls to 0.038 and MCC to 0.062.

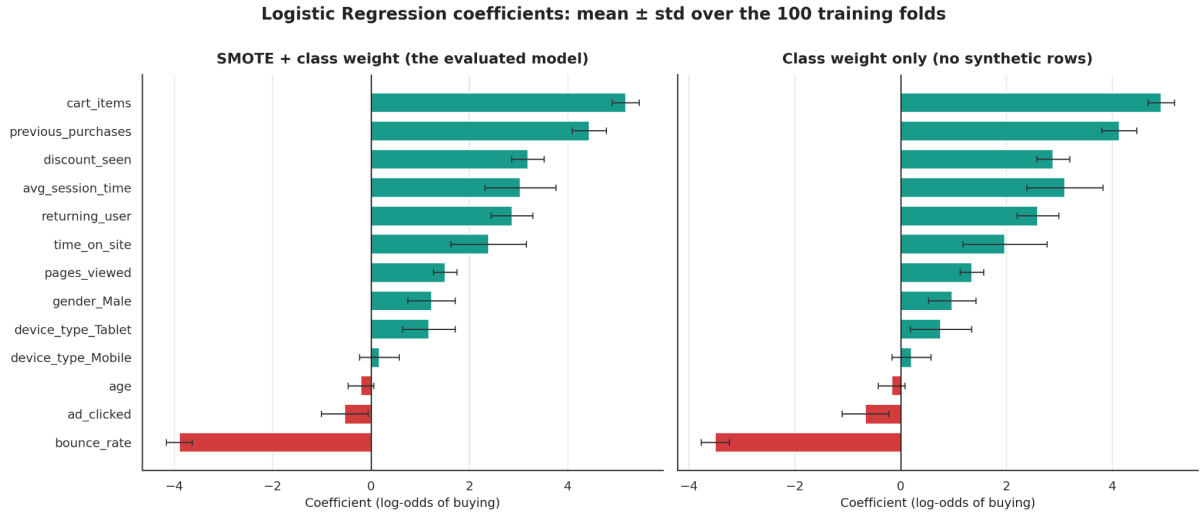
The main configuration is a defensible compromise, not a winner on every metric. SMOTE-only Logistic Regression has the highest MCC (0.539); class-weight-only Logistic Regression has the highest recall (0.977); SMOTENC plus class weights has the highest precision (0.365). The selected SMOTE-plus-class-weight setup retains high recall and strong PR-AUC while keeping the main model comparison consistent.

5.4 Logistic Regression Coefficients

Coefficients are for the probability of purchase, so a positive value raises purchase odds and a negative value lowers them. Continuous features are standardised, so a coefficient is the change in log-odds for a one-standard-deviation increase. The table compares the final all-data model with the mean and spread over the 100 training folds, with and without SMOTE.

Feature	Final fit (all data)	Fold mean ± std (SMOTE)	Fold mean (no SMOTE)	Direction
`cart_items`	+5.74	+5.18 ± 0.27	+4.92	Raises purchase odds
`previous_purchases`	+4.77	+4.43 ± 0.35	+4.13	Raises purchase odds
`discount_seen`	+3.51	+3.19 ± 0.33	+2.88	Raises purchase odds
`avg_session_time`	+3.46	+3.03 ± 0.72	+3.10	Raises purchase odds
`returning_user`	+3.21	+2.86 ± 0.43	+2.59	Raises purchase odds
`time_on_site`	+2.26	+2.38 ± 0.77	+1.97	Raises purchase odds

pages_viewed	+1.51	+1.51 ± 0.24	+1.35	Raises purchase odds
device_type_Tablet	+1.40	+1.17 ± 0.54	+0.76	Raises purchase odds
gender_Male	+1.26	+1.23 ± 0.48	+0.97	Raises purchase odds
device_type Mobile	+0.38	+0.17 ± 0.41	+0.20	Raises purchase odds
age	-0.28	-0.21 ± 0.26	-0.17	Lowers purchase odds
ad_clicked	-0.66	-0.54 ± 0.48	-0.67	Lowers purchase odds
bounce_rate	-4.06	−3.90 ± 0.27	-3.51	Lowers purchase odds



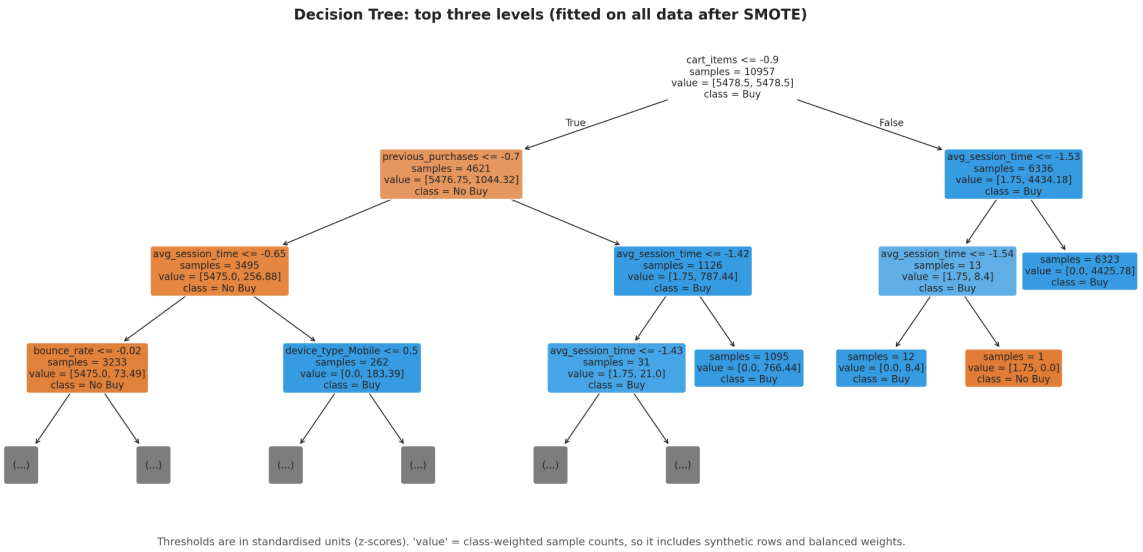
Across the coefficient analysis, cart_items and previous_purchases are the strongest positive purchase-associated signals. Discount exposure, average session time, returning-user status, time on site, and pages viewed also have positive coefficients in the fitted models; bounce_rate has the strongest negative coefficient. These are conditional predictive associations, not causal effects.

Coefficient signs for the stronger predictors are comparatively consistent across folds and with/without SMOTE. For age, ad_clicked, and device_type_Mobile, fold variability is large relative to the mean; these effects should not be presented as stable. Demographic coefficients require fairness review before any real-world use.

age, ad_clicked and device_type_Mobile

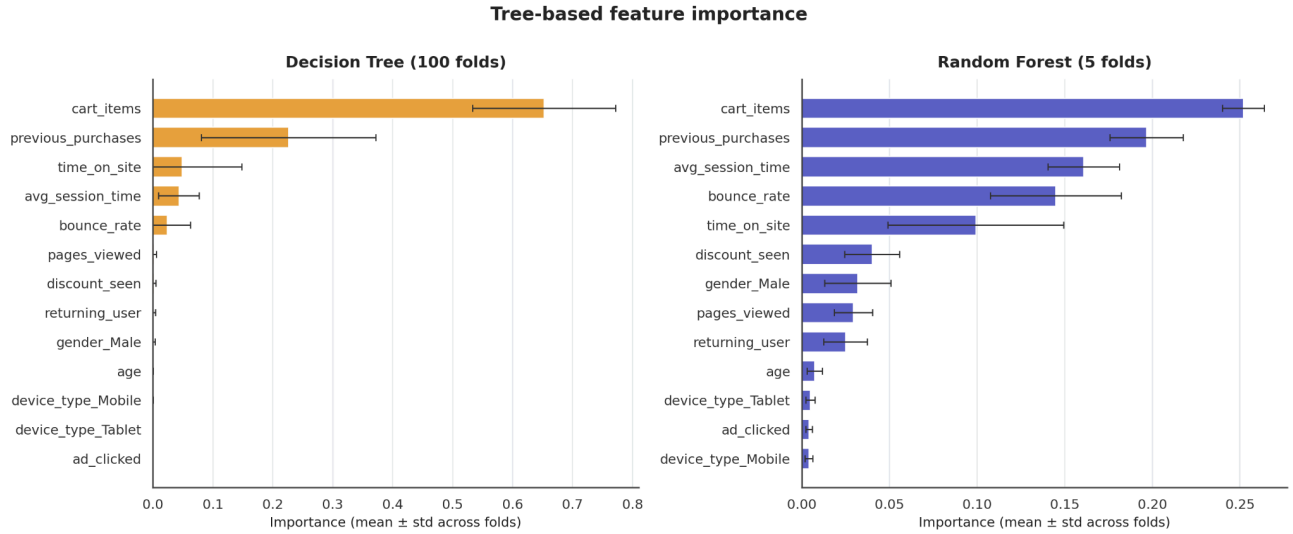
5.5 Tree-based Interpretation

The Decision Tree diagram and tree-based feature-importance plots provide complementary views of how the tree models use the session features. The diagram shows the split rules in one fitted Decision Tree, while the importance charts summarize feature use across fitted trees. These results describe the models' learned patterns; they do not establish causality.



The full-data Decision Tree uses cart_items for its root split (approximately $z \leq -0.9$), followed by features including previous_purchases and avg_session_time. Thresholds for standardized continuous variables are z-scores. Displayed counts reflect the resampled and class-weighted fitting data, not the original population. The plot illustrates one fitted tree's structure, not generalization performance.

The importance plots show means and standard deviations across fitted models: 100 Decision Tree fits and five Random Forest fits. The unequal number of fits means the error-bar estimates have different bases and should not be compared as if equally precise.



These are impurity-based importance values: they summarize each feature's contribution to reducing split impurity within the fitted model. They do not show whether an association is positive or negative, and they do not establish causality.

Feature	Decision Tree importance (100 fits)	Random Forest (5 fits)
`cart_items`	0.652 ± 0.119	0.252 ± 0.012
`previous_purchases`	0.226 ± 0.146	0.197 ± 0.021
`avg_session_time`	0.043 ± 0.034	0.161 ± 0.020
`bounce_rate`	0.024 ± 0.039	0.145 ± 0.037
`time_on_site`	0.049 ± 0.099	0.010 ± 0.050
`discount_seen`	0.002 ± 0.003	0.040 ± 0.016

The two panels use separate scales, so the values are best compared within each model, not treated as directly comparable effect sizes.

Random Forest distributes importance more broadly than the single Decision Tree. Its top features are cart_items (0.252), previous_purchases (0.197), avg_session_time (0.161), and bounce_rate (0.145). The Decision Tree concentrates importance in cart_items (0.652) and previous_purchases (0.226). Importances are model-specific contributions, not causal effects.

A lower overall importance does not necessarily mean a feature is irrelevant; it may help distinguish observations within a limited branch without accounting for much of the total split improvement.

The error bars show that the importance assigned to some features varies across cross-validation fits.

Agreement across methods

the table below ranks each feature under each method with 1= most influential:

Feature	Logistic Regression	Decision Tree	Random Forest	Average rank
cart_items	1	1	1	1.0
previous_purchases	2	2	2	2.0
avg_session_time	5	4	3	4.0
bounce_rate	3	5	4	4.0
time_on_site	7	3	5	5.0
discount_seen	4	7	6	5.7
pages_viewed	8	6	8	7.3
returning_user	6	8	9	7.7

Across Logistic Regression, Decision Tree, and Random Forest, cart_items and previous_purchases are consistently among the most informative variables. Session duration and bounce_rate also contribute. This cross-model agreement supports their relevance within this dataset, but does not show that changing these features would cause a purchase.

5.6 PCA Results

PCA on the 13 standardized transformed features shows no dominant component: PC1 explains 14.1%, PC2 10.9%, and PC3 8.1%, for 33.1% cumulatively. PC1 is dominated by time_on_site and avg_session_time (loadings about +0.71); PC2 by the Mobile and Tablet indicators (about +0.70 and −0.70); PC3 combines gender_Male, previous_purchases, discount_seen, cart_items, and age. PCA is unsupervised and does not measure predictive performance.

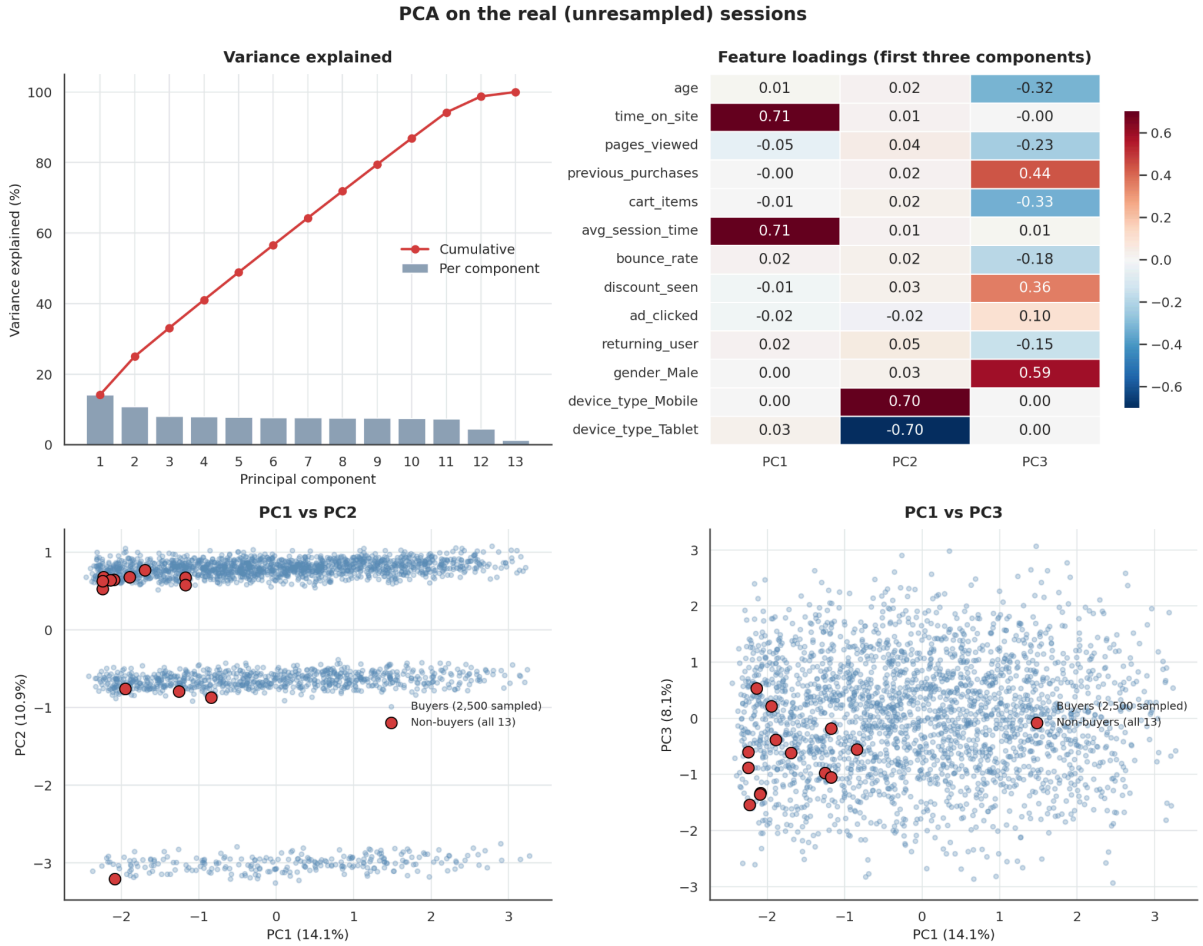
The tables below display the variance explained by each principal component and the loadings of the first three components

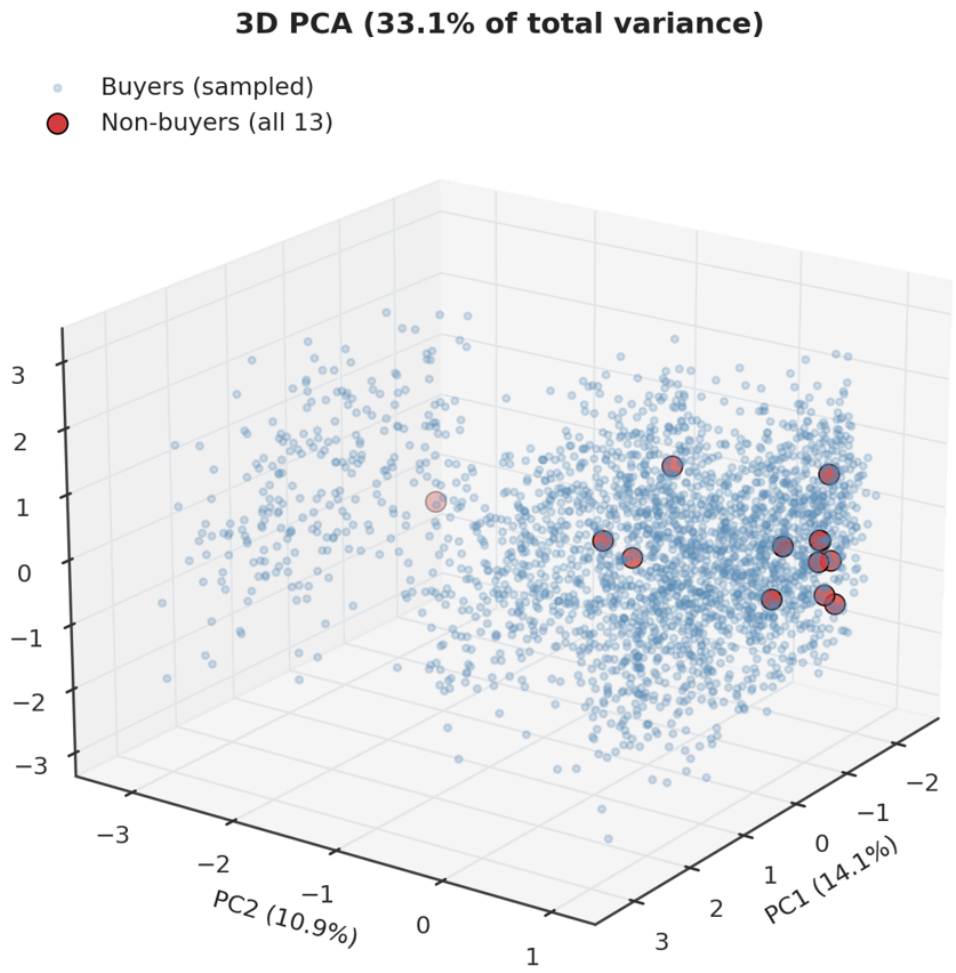
Component	Variance explained (%)	Cumulative variance (%)

PC1	14.15	14.2
PC2	10.87	25.0
PC3	8.10	33.1
PC4	7.92	41.0
PC5	7.82	48.9
PC6	7.73	56.6
PC7	7.69	64.3
PC8	7.60	71.9
PC9	7.55	79.4
PC10	7.45	86.9
PC11	7.31	94.2
PC12	4.53	98.7
PC13	1.26	100.0

Feature	PC1	PC2	PC3
age	+0.01	+0.02	-0.32
time_on_site	+0.71	+0.01	-0.00
pages_viewed	-0.05	+0.04	-0.23
previous_purchases	-0.00	+0.02	+0.44
cart_items	-0.01	+0.02	-0.33
avg_session_time	+0.71	+0.01	+0.01
bounce_rate	+0.02	+0.02	-0.18
discount_seen	-0.01	+0.03	+0.36
ad_clicked	-0.02	-0.02	+0.10
returning_user	+0.02	+0.05	-0.15
gender_Male	+0.00	+0.03	+0.59

device_type_Mobile	+0.00	+0.70	+0.00
device_type_Tablet	+0.03	-0.70	+0.00





Non-buyers are more concentrated in some lower-PC1 regions but overlap with buyers in the leading two- and three-component views. The plots therefore show partial concentration, not a cleanly isolated minority cluster. Overlap in a low-dimensional PCA projection does not establish that the full feature space lacks predictive signal.

6. Key Findings

6.1 Logistic Regression is the preferred model

It leads on the measures most relevant to the rare class: PR-AUC 0.783, MCC 0.512, recall 0.947 and ROC-AUC 0.999. At the default threshold it finds about 12.6 of 13 non-buyers per pass and misses about 0.4. Its precision of 0.286 means roughly three in ten flagged sessions are real non-buyers, about **170 times** the 0.17% base rate.

6.2 Accuracy is a misleading selection metric

All three models score between 99.6% and 99.9%, and the trivial all-buyer model scores 99.83%. Random Forest has the highest accuracy yet finds only 24% of

non-buyers. PR-AUC, recall, precision and MCC separated the models; accuracy did not.

6.3 Tree-based models were less suitable

The Decision Tree reached PR-AUC 0.115 ± 0.104, recall 0.367 and MCC 0.230 with large variation between splits, consistent with fitting a single tree on about ten actual non-buyers. Random Forest ranked well on ROC-AUC (0.991) and best on precision (0.432), but was unstable and missed most non-buyers (recall 0.242).

6.4 The threshold is a business choice

Moving the threshold from 0.40 to 0.95 trades recall from 1.000 to 0.708 and cuts alert volume from about 53 to about 16 sessions per pass. The right setting depends on the cost of a missed non-buyer against the cost of an unnecessary intervention..

6.5 Model choice is robust, SMOTE is not decisive for it

- Class weighting alone gave Logistic Regression its highest recall (0.977), yet lower precision (0.255).
- SMOTE only achieved the highest Logistic Regression MCC (0.539)
- SMOTENC plus class weighting produced the highest Logistic Regression precision (0.365), yet the lowest PR-AUC (0.711).
- SMOTE was particularly important for Random Forest. Without resampling, its minority recall and MCC collapsed to 0.038 and 0.062.

Logistic Regression led on PR-AUC in all four configurations. Its PR-AUC was nearly the same with and without SMOTE, whereas Random Forest collapsed without resampling. The practical consequence is that a simpler class-weight-only Logistic Regression is a legitimate alternative.

6.6 Behavioural drivers are consistent

Cart items, previous purchases, session duration, discount exposure, returning-user status and bounce rate are the main signals under Logistic Regression, Decision Tree and Random Forest alike. PCA shows that non-buyers overlap with buyers in the leading components, so they are identified through combinations of signals.

7. Recommendations

7.1 Use Logistic Regression as the primary screening model

Logistic Regression should be retained as the baseline. The saved pipeline contains the whole sequence (imputation, scaling, encoding, SMOTE during fitting, classification), so new sessions are processed exactly as in training

7.2 Treat the model as a prioritisation tool

Use predicted non-buyer probability to rank or flag sessions, not as an automatic, irreversible decision. Suitable uses include exit-intent messages, checkout support, product suggestions, low-cost incentives, remarketing and prioritising support. Because about seven in ten flags are false positives at 0.5, interventions should be cheap, reversible and unobtrusive.

7.3 Select the threshold according to intervention cost

The default threshold of 0.5 is useful for comparing models but should not automatically be used in production.,A suitable operational threshold depends on the business context.

Business objective	**Threshold**	**Expected behaviour**
Minimize missed non-buyers	0.40 to 0.50	Recall 0.97 to 1.00; about 46 to 53 sessions flagged per pass
Balanced screening	0.55 to 0.70	Recall about 0.89 to 0.94; about 33 to 41 sessions flagged
Act only on high-confidence cases	0.80 to 0.95	Precision 0.41 to 0.57; recall 0.71 to 0.88; 16 to 28 flagged

Fix the final threshold only after estimating the value of a successful intervention and the cost of contacting a customer who would have bought anyway.

7.4 Monitor the strongest signals

Track cart-item count, previous purchases, average session duration, time on site, discount exposure, returning-user status and bounce rate. Sessions with low cart activity, no purchase history, short duration and high bounce rate are natural candidates for assistance. These are predictive patterns to validate, not proof of cause.

7.5 Test interventions experimentally

The model identifies likely non-buyers; it does not show which action converts them.

Test actions such as a targeted discount against none, live-chat help, a checkout reminder or product recommendations in randomised A/B experiments before making any of them policy.

7.6 Recalibrate and monitor

Check probability calibration on new data, track recall, precision and alert volume over time, and retrain when behaviour shifts. Class weights and SMOTE distort predicted probabilities, so do not read them as literal purchase chances without calibration.

7.7 Review fairness before using demographic variables

`age` and `gender` are inputs to the model. Before real use, check error rates across groups and decide whether they should be included. In this dataset their effects are small.

8. Limitations

The following limitations qualify every result above. The first two are the most important.

Limitation	Why it matters	Mitigation
Only 13 non-buyers	Estimates depend on very few cases; a fold holds 2 to 3 of them, so metrics are noisy and may not generalise.	Collect more genuine non-buyer sessions; re-run the same pipeline.
Simulated data	Relationships may be artefacts of the simulation and do not describe a real platform.	Validate on real session logs.
Resampling	SMOTE builds synthetic rows from 10 to 11 real neighbours, so it cannot add genuinely new behaviour, and it distorts probabilities.	Compare against class weights alone (done); calibrate.
Calibration unverified	Predicted probabilities were not checked against observed frequencies.	Add calibration curves and recalibrate on new data.
No external	All estimates come from	Hold out time-based or

test set	cross-validation on the same 7,840 sessions.	independent data.
Association, not cause	Coefficients and importances show prediction, not the effect of changing a feature.	Run controlled experiments.
Importance caveats	Tree importances are model-specific and can be biased; Random Forest used only 5 fits.	Use permutation importance or SHAP in future work.
PCA is exploratory	It is unsupervised and low-dimensional; overlap is not proof of inseparability.	Rely on the supervised results for performance.
Threshold drift	Recall and alert volume may change when traffic mix changes.	Monitor and re-tune in production.
Statistics are descriptive	Fold scores overlap in training data, so standard devisations are not confidence intervals.	Report them as variability only.

Suggested next steps:

1. Obtain more non-buyer data. Even 100 genuine non-buyers would make every estimate far more reliable.
2. Validate externally on real or at least independently generated sessions, preferably split by time.
3. Calibrate the probabilities (for example isotonic or Platt scaling) and report calibration curves.
4. Add richer interpretation with permutation importance or SHAP and a partial-dependence view.
5. Try more models such as gradient boosting, with a focus on stability on the minority class.
6. Run a pilot A/B test of a low-cost intervention at a chosen threshold.

9. Conclusion

This project developed a leakage-aware workflow for predicting purchase outcomes in a highly imbalanced e-commerce dataset. Of 7,840 labelled sessions, only 13 were non-buyers, making rare-class evaluation the central analytical challenge.

Preprocessing and SMOTE were performed within training folds, and three classifiers were compared using repeated stratified five-fold cross-validation (20 repeats; 100 evaluations per model). Logistic Regression was preferred, with mean

PR-AUC 0.783, MCC 0.512, recall 0.947, and precision 0.286 under the main configuration.

Threshold analysis showed how the selected model can trade alert volume against missed non-buyers. Sensitivity analysis confirmed that Logistic Regression led on PR-AUC across all four imbalance strategies. Cart activity, purchase history, session engagement, and bounce rate were recurrent predictive signals; these associations are not causal evidence.

The results are exploratory because the data are simulated, the minority class contains only 13 observations, and no independent external test set was available. Before operational use, validate on substantially more real non-buyers, assess calibration and fairness, monitor performance, and test interventions experimentally. Within these limits, the project provides a structured and interpretable basis for further evaluation of non-conversion screening.